



1. Problem Title & Link

Title: LeetCode 1657 : Determine if Two Strings Are Close

Link: [LeetCode 1657 - Determine if Two Strings Are Close](#)

2. Problem Statement (Short Summary)

You are given two strings:

word1

word2

Two strings are considered **close** if we can transform one string into another using:

Operation 1:

Swap any two existing characters

Operation 2:

Transform every occurrence of one character

into another existing character and vice versa

In simple terms:

Check whether both strings can become identical after rearranging and swapping character frequencies.

3. Examples (Input → Output)

Example 1

Input:

word1="abc"

word2="bca"

Output:

true

Explanation:

Rearranging characters makes them equal.

Example 2

Input:

word1="a"

word2="aa"

Output:

false

Explanation:

Lengths differ.

**Example 3**

Input:

word1="cabbba"

word2="abbccc"

Output:

true

Explanation:

Frequency:

word1:

a→1

b→3

c→2

word2:

a→1

b→2

c→3

Frequencies can be rearranged.

4. Constraints

1 <= word1.length,

word2.length <=10⁵

word1 and word2 contain lowercase English letters

Special restrictions:

- Characters cannot be created
- Characters cannot disappear

5. Core Concept (Pattern / Topic)

HashMap / Frequency Counting

6. Thought Process (Step-by-Step Explanation)

Brute Force

Try all transformations:



1. Generate possible swaps
2. Generate character mappings
3. Compare results

Time:

Exponential

Impossible for large inputs.

Optimized Thinking

Observation:

Two conditions must be true:

Condition 1:

Both strings must contain exactly the same characters.

Example:

abc

abd

Impossible because:

d does not exist in word1

Condition 2:

Frequency distributions must match.

Example:

a→1

b→2

c→3

and

x→1

y→2

z→3

can be transformed.

Algorithm:

1. Count character frequencies
2. Compare character sets
3. Compare sorted frequency values

7. Visual / Intuition Diagram (ASCII Diagram)

Input:



```
word1="cabbba"
```

```
word2="abbccc"
```

```
Frequency:
```

```
word1
```

```
a→1
```

```
b→3
```

```
c→2
```

```
word2
```

```
a→1
```

```
b→2
```

```
c→3
```

```
Sort frequencies:
```

```
[1,2,3]
```

```
[1,2,3]
```

```
Match:
```

```
true
```

8. Pseudocode (Language Independent)

```
count1=frequency(word1)
```

```
count2=frequency(word2)
```

```
if keys(count1)
```

```
!=
```

```
keys(count2):
```

```
return false
```

```
if sorted(values(count1))
```

```
!=
```

```
sorted(values(count2)):
```



```
return false
```

```
return true
```

9. Code Implementation

Python

```
from collections import Counter
```

```
class Solution:
```

```
    def closeStrings(  
        self,  
        word1,  
        word2):  
  
        count1=Counter(word1)  
        count2=Counter(word2)  
  
        return (  
            set(count1.keys())  
            ==  
            set(count2.keys())  
            and  
            sorted(count1.values())  
            ==  
            sorted(count2.values())  
        )
```

Java

```
class Solution {  
  
    public boolean closeStrings(  
        String word1,  
        String word2) {  
  
        int[] count1=
```



```
        new int[26];

int[] count2=
    new int[26];

for(char c:
    word1.toCharArray()){

    count1[c-'a']++;
}

for(char c:
    word2.toCharArray()){

    count2[c-'a']++;
}

for(int i=0;i<26;i++){

    if(
        (count1[i]==0
        &&
        count2[i]!=0)

        ||

        (count1[i]!=0
        &&
        count2[i]==0)
    ){

        return false;
    }
}
```



```
Arrays.sort(count1);
Arrays.sort(count2);

return Arrays.equals(
    count1,
    count2
);
}
```

10. Time & Space Complexity

Time Complexity:

$O(n \log n)$

Reason:

- Frequency counting $\rightarrow O(n)$
- Sorting frequencies $\rightarrow O(26 \log 26)$

Since alphabet size is fixed:

Practically $O(n)$

Space Complexity:

$O(1)$

Reason:

Frequency arrays remain fixed size.

11. Common Mistakes / Edge Cases

1. Checking only frequencies

Wrong:

word1="abc"

word2="abd"

Frequencies match but characters differ.

2. Forgetting character existence check

3. Using only sorted strings

Wrong:

Sorting ignores operation rules



4. Missing identical strings case

word1="abc"

word2="abc"

12. Detailed Dry Run (Step-by-Step Table)

Input:

word1="cabbba"

word2="abbccc"

Character	Word1 Count	Word2 Count
a	1	1
b	3	2
c	2	3

Sorted frequencies:

Word 1	Word 2
[1,2,3]	[1,2,3]

Result:

true

13. Common Use Cases (Real-Life / Interview)

Anagram pattern analysis

Data transformation validation

Character mapping systems

Text normalization

14. Common Traps (Important!)

1. Forgetting same character set condition
2. Using sorted strings only
3. Ignoring frequency distribution
4. Missing duplicate counts

15. Builds To (Related LeetCode Problems)

Progression:



- LC 1657 → Determine if Two Strings Are Close
- LC 49 → Group Anagrams
- LC 242 → Valid Anagram
- LC 451 → Sort Characters By Frequency
- LC 438 → Find All Anagrams in a String

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	Exponential	High	Impossible
HashMap + Sort	$O(n \log n)$	$O(n)$	Good
Frequency Array	$O(n)$	$O(1)$	Optimal

17. Why This Solution Works (Short Intuition)

The operations only allow rearranging characters and exchanging frequencies between existing characters. Therefore, both strings must have the same character set and identical frequency distributions.

18. Variations / Follow-Up Questions

1. Support uppercase characters
2. Support Unicode strings
3. Find minimum operations required
4. Return transformation sequence
5. Check similarity among multiple strings
- 6.